Technical Report — Fact Attack: Multilingual NLI over Swiss Voting Booklets

- **Track:** Track 2A — OST: Multilingual Natural Language Inference over Swiss Official Voting Booklets
- **Event:** Hack Apertus Online (October 1–16, 2026)
- **Team:** Team Fact Attack 2026 — Josip Jukic, Felipe Wüthrich, Sheyla Sampietro, Andrea Petretta
- **Repository:** GitHub · **Experiment log:** docs/experiments.md
- **Demo:** Streamlit app (`make web`: claim check with highlighted evidence pages, benchmark dashboard) and the entailment CLI (`python -m src --input cases.jsonl --output predictions.jsonl`, the Docker image's entrypoint, see 2.5)

1. Summary

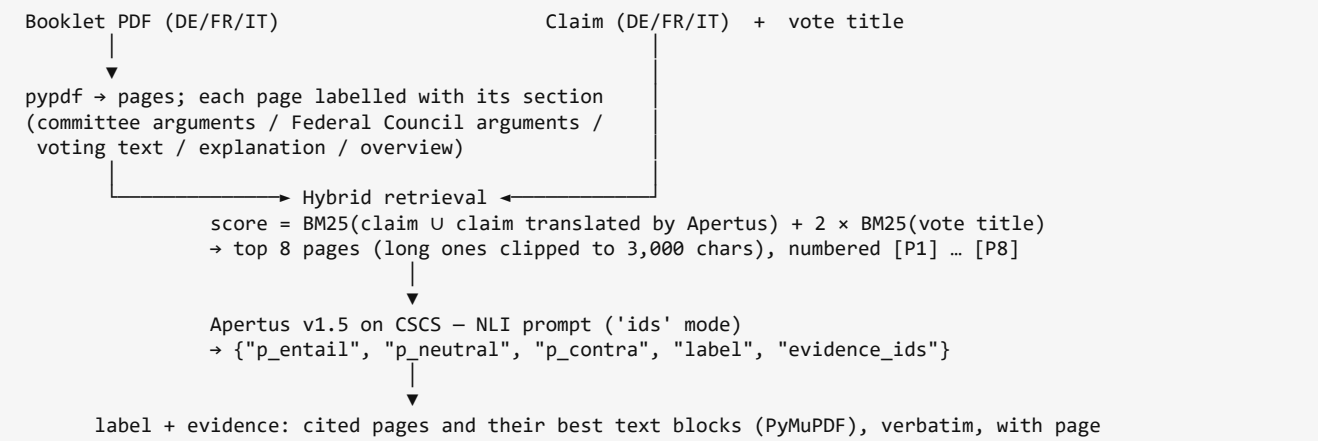
Swiss voters receive an official booklet for every federal vote, and campaigns make claims about it that are hard to check. We built a multilingual claim-verification system on **Apertus v1.5**: given a booklet (German, French or Italian) and a claim (any of the three languages), it classifies the claim as **Entailment (0)**, **Neutral (1)** or **Contradiction (2)** and returns the booklet pages that justify the decision, together with token usage and inference time.

On **booklets the system never saw** (our holdout: 90 claims on the 27 September 2026 vote, labels checked by a second person, run once), it reaches **macro-F1 0.93–0.94** and **evidence Hit@5 0.83–0.85** on the advanced task (booklet PDF) and **0.966** on the beginner task (reference text). On the whole official dataset (v1.1, 1,488 pairs, 20 voting dates in 5 date-grouped folds) the same configuration scores **0.943 / Hit@5 0.869** and **0.984**, at 3,866 / 1,732 input tokens and 62–76 output tokens per case, with processing time outside the LLM calls in the millisecond range. All scores come from the organisers' `evaluate.py` on the official CLI's output.

The official score also weighs evidence, processing time and tokens (relative to the best team), so after the organisers' Q&A we re-tuned for it (section 2.6). The two team members first optimised separately and then merged their lines part by part, each choice decided by paired runs on the same cases: 8 instead of 12 pages (−31 % input tokens, −0.003 F1 on dev), evidence quoted from PyMuPDF pages and text blocks (Hit@5 0.73 → 0.81 on the same predictions), BM25 retrieval with the claim and its translation into the booklet language (macro-F1 0.930 → 0.945 on test), a start-up and parsing path that keeps the scored processing time in milliseconds, and, last, beginner-task references cut to the passages that match the claim (−30 % input tokens at equal macro-F1).

The central finding for the challenge's research question: **sending a dozen well-chosen pages beats sending the whole booklet** — 0.946 vs. 0.730 macro-F1 on the same 150 test pairs, with ~10× fewer input tokens.

2. Architecture



2.1 Document processing

`pdf_parser.py` extracts the text of every page with pypdf and detects section headings in all three languages ("Argumente Initiativkomitee", "Arguments du Conseil fédéral et du Parlement", "Il testo in votazione", …). A heading applies to its page and the following pages until the next heading, and table-of-contents entries (heading followed by a page number) are ignored. Section labels matter because booklets deliberately contain opposing viewpoints: the model has to know whether a page states the committee's or the Federal Council's position. The same pass also reads every page with PyMuPDF (text and text blocks) for the evidence quotes (2.3). Booklets that are not in the image are parsed by a process pool at start-up, one booklet after the other in the order of their cases; the parse of the 60 dataset booklets is baked into the image.

2.2 Context selection: hybrid retrieval

`retriever.py` scores every page of the booklet with normalised BM25 against the claim plus twice the normalised BM25 against the vote title, and keeps the top 8 pages (12 before the Q&A, see 2.6). Pages longer than 3,000 characters are clipped to the window that best matches claim and vote title (plus the page heading).

- The **vote title is written in the booklet's language**, so it anchors the right proposal even when the claim is in another language; lexical matching of the claim alone fails for cross-lingual pairs.
- Our first version filtered hard to one detected proposal. That dropped the per-proposal overview pages at the front of the booklet, where the gold passage sits for ~11 % of pairs, and chose the wrong proposal for another ~10 %. Booklet-wide scoring raised gold-section recall on the dev split from 0.74 to 0.93.
- **Query translation** (`RETRIEVAL_QUERY_MODE=union`): for a cross-lingual claim, Apertus first translates it into the booklet language (one short request) and BM25 scores the pages with both versions; the NLI prompt keeps the original claim. Paired against the original query: 48 fixed / 25 broken labels over three comparisons ($p \approx 0.01$).
- **Speaker boost:** if the claim names a side ("the initiative committee argues …", "selon le Conseil fédéral …"), the two best argument pages of that side are always among the selected pages, replacing the lowest-ranked others. Without it, the committee's own page was missing in 15 of 36 committee-claim errors.

2.3 Apertus as the NLI model

The selected pages are numbered `[P1]…[P8]`. The system prompt defines the three labels, tells the model to judge only against the passages (not world knowledge), to answer Neutral for topics or details the passages do not address, to check claims attributed to an actor only against that actor's text, and to read "recommends rejecting X" as supported when the Federal Council and Parliament recommend "No". Apertus answers with a small JSON object: three confidences, the label, and the ids of the pages that justify it.

- **Evidence by id** guarantees verbatim booklet text with the correct page number and avoids paraphrased quotes.
- **Short answers**: ~55 output tokens instead of ~150 for a JSON answer with reasoning and quotes, which more than halves inference time (p95 5.5 s → 1.8 s on the dev split) at equal or better F1.
- **Evidence items:** up to three cited pages as PyMuPDF reads them, then the two text blocks of the cited pages that best match claim and vote (at most five items). Hit@5 counts an item only if it lies inside the gold passage or contains it; the gold passages follow PyMuPDF's text, so the same blocks built from pypdf or pypdfium2 text reach only 0.80 / 0.73 instead of 0.87 on dev. The PyMuPDF pages are stored with the parse, so no PDF is opened while cases are answered.
- For **Neutral**, the evidence list is empty. If the model predicts 0 or 2 without citing a page, the top-ranked page is used.
- **Decision rules:** in ~2 % of answers the label and the stated confidences disagree. The label is right 3× as often, so it is kept (Rule 0) — except a Neutral label next to a confident other relation, which was right in only 1 of 18 cases and follows the confidences (Rule 0b). Otherwise label and confidences agree and are used as given.

2.4 Beginner task

The reference string (up to 25k characters) is split into passages of ≤ 700 characters at sentence boundaries, numbered and sent with the same prompt, so evidence is a specific passage rather than the whole reference. A reference longer than 6,000 characters is cut to its opening passage plus the passages that best match the claim (BM25 with the claim and, for cross-lingual pairs, its translation), in document order, up to 6,000 characters. On all 1,488 beginner cases this saves 30 % of the input tokens at equal macro-F1 (0.981 → 0.984; 8 fixed / 3 broken claims, against 2 changed labels when the unchanged system runs twice); on the holdout 0.955 → 0.966 at −13 %. The length of a reference is never used as a hint for its label.

2.5 CLI and output format

`python -m src --input cases.jsonl --output predictions.jsonl` (the Docker entrypoint) reads the official case format (JSON or JSONL; a case has `booklet.path` + `vote` for the advanced task or `reference.text` for the beginner task) as well as rows of the OST dataset on Hugging Face (`claim`, `reference_string`, `booklet_url`, `vote`; also Parquet or CSV), and writes one record per case:

```
{"id": "case-0042", "label": 2, "label_name": "contradiction",
 "evidence": [{"page": 7, "text": "<page 7 as PyMuPDF reads it>"}, {"page": 7, "text": "<best text block>"}, ...],
 "metrics": {"input_tokens": 3983, "output_tokens": 75, "inference_time_ms": 2053}}
```

Evidence is empty for Neutral, and its page is `null` for a reference text. The claim language is taken from the case (or detected), and a failing case is reported on stderr and returned as a valid Neutral record instead of aborting the batch; the exit code is 0. `python -m src predict` checks a single claim.

2.6 Evaluation contract and efficiency

The organisers run the image as `--input /data/cases.jsonl --output /output/predictions.jsonl` with `BASE_URL` (a token-counting proxy) and `API_KEY` injected, `/data` read-only. The score is `0.6 · S_A + 0.4 · S_B`, where S_A = 50 % macro-F1 + 20 % evidence Hit@5 + 15 % processing time + 15 % tokens and S_B = 70 % macro-F1 + 15 % time + 15 % tokens; time and tokens count as best ÷ own over all teams, and processing time is the wall clock minus the time with an Apertus request in flight.

- **Contract:** `BASE_URL` / `API_KEY` take precedence over our older variable names; the parse cache of the 60 dataset booklets is baked into the image read-only and new entries go to `/tmp`; a CI step runs the image exactly this way.
- **Tokens:** 8 pages instead of 12 (−31 % input tokens); beginner references cut to the matching passages (−30 %, 2.4); a condensed prompt, narrower page windows and fewer legal-text pages cost F1 and were rejected (5.3).
- **Processing time:** a standard-library HTTP client instead of the openai SDK (~1 s less start-up); cases run four at a time, so parsing and retrieval of one case overlap with requests of others; uncached booklets are parsed by a process pool started before anything else, and cases are processed booklet by booklet. The CLI reports its own estimate of the scored time from module entry: ~6 ms per case with cached booklets (test split), ~22 ms on new booklets (104 ms before the pool).
- **Robustness:** a byte order mark, broken or non-object lines in the case file, unreadable PDF pages (pypdfium2 fallback), API errors (retries, then one JSON-constrained last attempt) and a stuck parser each cost at most their own case, which is returned as a valid Neutral record; the exit code stays 0.

3. Use of Apertus

- **Model:** `swiss-ai/Apertus-v1.5-70B-thinking` (the model our CSCS key is authorised for; the thinking phase is not triggered by our prompt, answers start directly with the JSON object).
- **Where it runs:** CSCS inference endpoint (`https://api.inference.cscs.ch/v1`), OpenAI-compatible chat completions, temperature 0, retries with exponential backoff.
- **Role:** Apertus is the only model in the pipeline. It translates cross-lingual claims for retrieval, performs the NLI classification and selects the evidence pages; retrieval is lexical (BM25) and runs locally. Apertus 8B v1.5 was used only for a comparison run (5.3); no other model was used for development or evaluation.

4. Data and evaluation protocol

- **Dataset:** `OSTswiss/MNLIoverSwissVotingBooklets` — 1,495 human-annotated claim/reference pairs from 60 booklets (2020–2026), balanced labels, ~66 % cross-lingual pairs.
- **Booklets:** downloaded from `bk.admin.ch` via the `booklet_url` field (`python -m src.hf_dataset`).
- **Split by voting date**, so all three language versions of a booklet stay on the same side: **dev** (1,093 pairs, 15 dates) and **test** (402 pairs, 5 dates). Test served several before/after decisions, so it is development data by now, not a holdout.
- **Official scorer:** cases from the starter repo's `prepare_cases.py` (dataset v1.1), scored with its `evaluate.py`. Final measurements run the official CLI on **all 1,488 pairs per task in five folds grouped by voting date** (`scripts/run_decision.py`, which forces the submission settings of `.env.example`), and report per fold, language pair and class, macro-F1 on de-duplicated pairs (38 % of the rows are exact duplicates) and a bootstrap over voting dates. Running the unchanged system twice changes 3 (advanced) / 2 (beginner) of 1,488 labels: the endpoint is nearly deterministic, so paired label changes measure the effect of a change.
- **Holdout on unseen booklets:** the jury evaluates on booklets outside the dataset. We wrote 90 claims (30 per label, 10 per language pair, claim types in the dataset's proportions) on the 27 September 2026 booklets before the system ever ran on them; a second team member checked every label (90/90 agree). It was run once, at the end. (An earlier set of 36 claims on the same booklets was used for two decisions and later lost.)

5. Results

5.1 Final configuration — official CLI and scorer

Evaluation set	Task	Macro-F1	Hit@5	Input / output tokens	Apertus time (mean / p95)
Holdout, unseen booklets (90 pairs, run once)	Advanced	**0.933–0.944**	**0.833–0.850**	3,863 / 67	–
	Beginner	**0.966**	–	1,758 / 60	–
All 1,488 pairs (5 date folds)	Advanced	**0.943** (folds 0.919–0.977)	**0.869**	3,866 / 76	1.9 s / 3.0 s
	Beginner	**0.984** (folds 0.968–0.990)	–	1,732 / 62	1.1 s / 1.3 s
Former test split (398 pairs)	Advanced / Beginner	0.942 / 0.982	0.833	3,979 / 1,741 input	–

The holdout's advanced range comes from two runs with identical settings (one label differs). Its errors are cross-lingual (de→fr, fr→de). Earlier milestones on the test split: 12 pages with whole cited pages 0.945 / Hit@5 0.341 at 5,598 tokens; 8 pages 0.930 / 0.727 at 3,877; + PyMuPDF evidence and query translation 0.945 / 0.839. Josip's comparison on all 1,488 advanced cases (k=12): baseline 0.948 / Hit@5 0.47, with query translation (R2) 0.956 / 0.77.

5.2 Full document vs. selected context (advanced, same 150 test pairs)

Context	Prompt	Macro-F1	Input tokens	Output tokens	Latency p95
Full booklet	JSON	0.730	59,517	202	43.7 s*
Proposal filter, top 5 pages	JSON	0.795	3,408	157	36.2 s*
Hybrid, top 10 pages	ids	0.926	6,023	55	3.8 s
Final (12 pages, clipping, speaker boost, decision rules)	**ids**	**0.946**	5,713	55	2.8 s

* measured while three runs shared the endpoint.

The full booklet (up to ~70k tokens) is both the most expensive and the least accurate option.

5.3 Ablations on the dev split

Change	Effect	Decision
`ids` answer instead of JSON with reasoning + quotes	beginner 0.973 → 0.980, advanced 0.786 → 0.797, p95 5.5 s → 1.8 s	adopted
Hybrid retrieval instead of proposal filter	advanced 0.797 → 0.862	adopted
Viewpoint and recommendation rules in the prompt	advanced 0.862 → 0.870, beginner 0.980 → 0.993	adopted
Decision-Rule 0: keep the label when it disagrees with the confidences	44 fixed / 13 broken over all saved runs; test 0.920 → 0.928	adopted
Clip pages > 3,000 characters to the best-matching window	test 0.925 → 0.928, input tokens 5,725 → 4,696 (p95 12.7k → 7.4k)	adopted
12 instead of 10 pages (after clipping)	all 1,495 pairs 0.912 → 0.931 (51 fixed / 24 broken); +~890 input tokens	adopted
Speaker boost (2.2)	all pairs 0.931 → 0.940 (22 fixed / 8 broken), committee-claim errors 36 → 27, no extra tokens	adopted
Decision-Rule 0b (2.3)	all pairs 0.940 → 0.945 (8 fixed / 1 broken)	adopted
600-character passages instead of pages (n = 450, paired)	0.831 vs. 0.908; better evidence precision, fewer tokens	rejected
Thinking before answering (prefilled reasoning marker)	dev 450: 0.908 → 0.929 (20 fixed / 11 broken); ~1,300 output tokens and ~23 s per claim instead of 59 and ~2 s; a 300-token budget is worse	rejected (efficiency)
Translate cross-lingual claims into the booklet language first	dev 0.908 → 0.919, test 0.925 → 0.923 (14 fixed / 11 broken); +0.6 s	rejected (noise)

Change	Effect	Decision
Other speaker fixes (verification pass, hiding the other side, few-shot, speaker hint) and answer formats (label-only with logprobs, reasoning sentence, confidence thresholds, numerical override, meta-classifier)	each broke at least as many answers as it fixed (verification pass: 0 / 24)	rejected
Apertus 8B instead of 70B (450 dev pairs)	0.851 vs. 0.908 F1, p95 1.2 s vs. 4.5 s	70B kept (quality first; Apertus latency is not scored)
Evidence: cited pages in pieces + best third of further pages (pypdf text)	test Hit@5 0.341 → 0.727	superseded by PyMuPDF evidence
8 instead of 12 pages (dev 300, official scorer)	F1 0.937 → 0.934, Hit@5 0.70 → 0.76, input tokens 5,370 → 3,721; k=7 / 6: F1 0.924 / 0.904; test F1 0.945 → 0.928	adopted (tokens are scored)
Condensed system prompt (`PROMPT_SHORT`)	dev 300 B 0.980 → 0.850, A 0.937 → 0.781, 6–13 % fewer input tokens	rejected
Evidence: PyMuPDF pages + 2 best text blocks (same predictions)	dev 300 0.792 → 0.872, test 0.727 → 0.813	adopted
Retrieval with claim + translation (`union`, paired real runs)	dev 300 0.934 → 0.944, test 0.930 → 0.945, all 1,488 0.948 → 0.956; ~3 % more tokens	adopted
Answer constrained by a JSON schema (`response_format`)	dev 150 0.933 → 0.833, Hit@5 0.78 → 0.69; no unparseable answer to fix (0 in > 4,000)	rejected
Drop low-value pages from the context (tables of contents, short pages, voting text)	dev 300 0.927 → 0.917 / 0.910, no gain on the new booklet	rejected
Paired runs on all 1,488 pairs per task (5 date folds; unchanged system run twice: 3 / 2 labels change):		
E1: at most 2 legal-text pages, freed places refilled	A 0.943 → 0.937 (11 fixed / 19 broken); legal-text claims lose context; −3 % tokens	rejected
E2: beginner references > 6,000 chars cut to the matching passages	B 0.981 → 0.984, −30 % input tokens; holdout 0.955 → 0.966	adopted
E3: page windows of 2,000 instead of 3,000 characters	A 0.943 → 0.934 (6 / 18), all classes worse; −9 % tokens	rejected
E4: evidence blocks ranked with the claim and its translation (offline replay)	Hit@5 0.869 → 0.873, not consistent across folds	rejected

Full tables: docs/experiments.md and docs/independent_experiments.md. Runs on 150 pairs carry about ±3 F1 points of sampling noise; close decisions were re-run on 450 pairs, the last ones on all pairs. A proxy measured without the model (is the gold page in the context, is the gold text still visible) predicted gains for E1 and E3 that the real runs did not show.

5.4 What we learned about Apertus

- **Selected context beats more context.** With the whole booklet (~60k tokens) Apertus 70B reaches 0.73 macro-F1, with 10 selected pages 0.93. We found no position effect ("lost in the middle"): the full-document runs fail even when the answer is in the first third of the booklet, so the degradation is general distraction.
- **Apertus is over-confident.** Its self-reported confidences are 0 or 1 in 97–99 % of cases, and the probability of the label token is ~1.0 for wrong answers as well as for correct ones (41 wrong vs. 40 correct dev answers). Confidence thresholds cannot filter its errors.
- **Speaker attribution was its main weakness — mostly a retrieval problem.** Claims attributed to the committee failed in 13.4 % of cases (2.2 % for Federal Council claims), and no prompt change helped. The speaker boost and Rule 0b brought them down to 7.4 %, the same rate as claims without a speaker.
- **Thinking helps, at a high price.** Letting the thinking model reason before its answer fixes half of the remaining dev errors (+0.021 F1 on dev 450), but needs ~22× the output tokens and ~11× the latency. A prompt instruction alone does not make it think first; the reasoning marker has to be prefilled. Selective thinking saves little (it is needed on ~3/4 of claims), and a short hard budget is worse.

- **Cross-lingual asymmetry.** Over all 1,495 pairs, French claims against German booklets are the hardest pair (0.880), French claims against Italian booklets the easiest (0.986). Apertus 8B is ~3.5× faster but 6 F1 points weaker.

6. Limitations

- **Remaining errors** (advanced, 85 of 1,488 pairs): Entailment → Contradiction 34, Entailment → Neutral 20, Contradiction → Neutral 19, Contradiction → Entailment 8, Neutral → Contradiction 3. Missing information is rarely read as a contradiction. Claims about the consequences of a yes fail most often (15.5 %, against 3.6 % in the beginner task, where the passage is given): their errors are mostly page selection.
- **Evidence:** Hit@5 is 0.87 on all pairs. Of the 130 misses, 55 cite another page than the annotated one, 39 are Neutral predictions (no evidence) and 36 cite the gold page without a matching item among the first five.
- **Unseen booklets:** the holdout (0.93–0.94 advanced, 0.966 beginner) covers one vote with two proposals and 90 claims written by us (± ~0.05); its errors are cross-lingual advanced claims. The jury's set may differ in layout and claim style.
- **License:** the evidence quotes need PyMuPDF (AGPL-3.0), so the image as a whole is distributed under the AGPL.
- **Development data:** all 20 dates of the dataset, including the former test split, have been used for decisions; only the holdout was not.
- **Layout:** committee pages headed only by a slogan stay unlabelled; tables and charts are read as plain text.
- **Processing time and tokens** are measured by the organisers' proxy; our own numbers are estimates (client-side token counts, in-flight clock). Booklets outside the 60 dataset booklets are parsed at run time (in parallel).
- **Not political advice:** outputs describe the relationship between a claim and the official booklet only.

7. Reproducibility

```
docker build --platform linux/amd64 -t fact-attack .          # pre-parses the 60 dataset booklets
docker run --rm -e BASE_URL -e API_KEY -v "$PWD/cases:/data:ro" -v "$PWD/output:/output" \
  fact-attack --input /data/cases.jsonl --output /output/predictions.jsonl
make test                                                      # unit tests, no API calls
```

Cases come from the starter repo's `prepare_cases.py` (or `scripts/prepare_competition_cases.py` for all pairs with date folds, `scripts/build_holdout.py` for the holdout) and are scored with its `evaluate.py`. `scripts/run_decision.py` runs the submission configuration through the official CLI and keeps predictions, scores and settings in `results/generalization/`; `scripts/report_decision.py` compares two runs. Repeated runs differ by a few pairs.

Settings: `.env.example` lists every setting with its default (a test keeps it equal to `src/config.py`), among them `NLI_TOP_K` (`8`), `PAGE_MAX_CHARS` (`3000`), `PROMPT_MODE` (`ids`), `RETRIEVAL_QUERY_MODE` (`union`), `EVIDENCE_POLICY` (`raw_pages_and_blocks`), `REFERENCE_MAX_CHARS` (`6000`), `NLI_WORKERS` (`4`).

8. Next steps

1. The remaining Entailment → Contradiction errors (34 of 85 advanced errors) and the cross-lingual errors on unseen booklets, both mostly with the gold page in the context; reasoning at lower cost (thinking: +0.02 F1 at ~11× latency).
2. A larger holdout over several new votes, labelled independently, to narrow the ± 0.05 on unseen booklets.

License

Code: Apache License 2.0 (LICENSE). Documentation, including this report: Creative Commons Attribution 4.0 (CC-BY-4.0), as required by the Hack Apertus terms (section 6). The image includes PyMuPDF (AGPL-3.0), so the image as a whole is distributed under the AGPL-3.0, with this public repository as its complete source.